Laporan M1 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

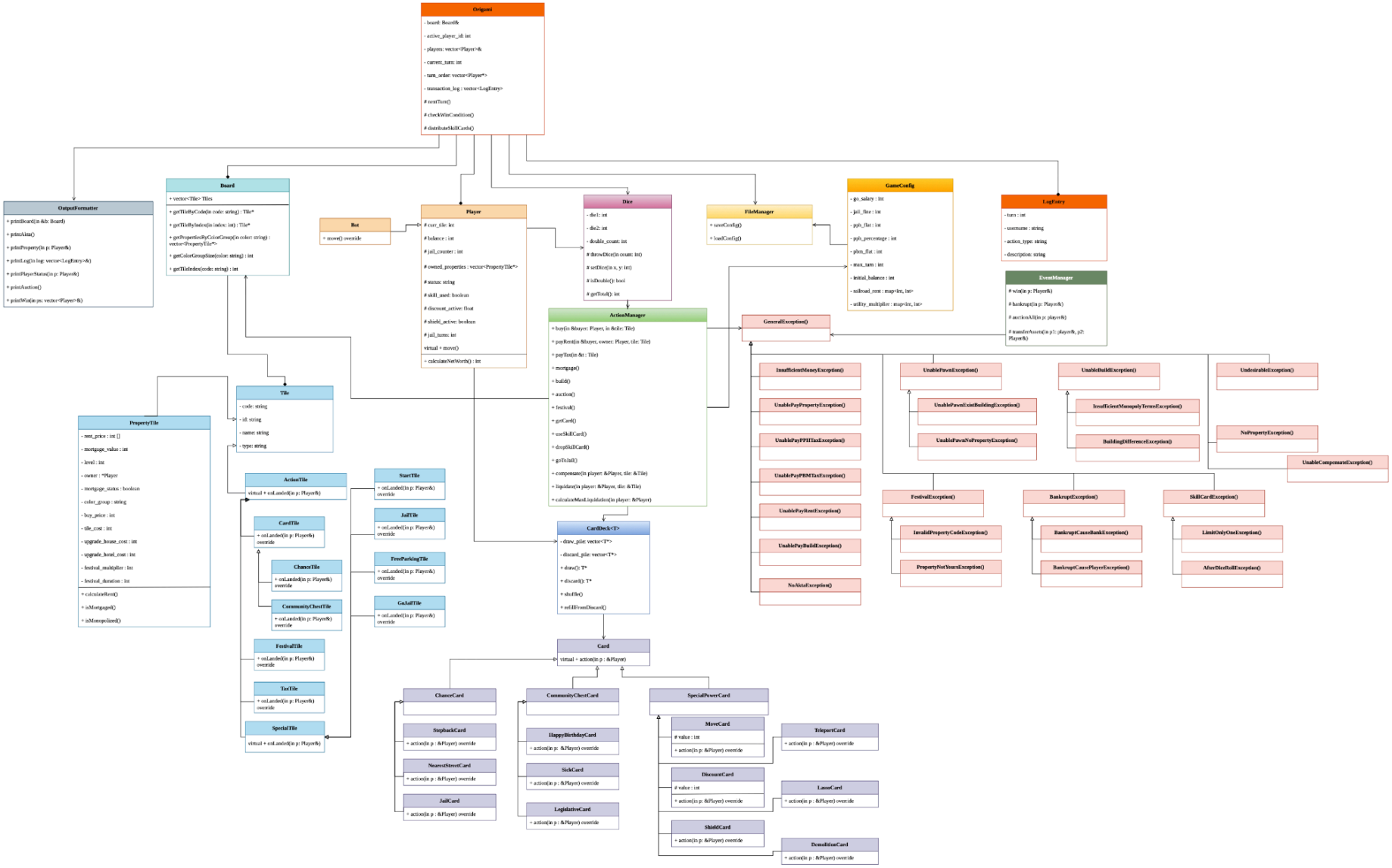
Kode Kelompok : ORI
Nama Kelompok : origami

1. 13524019 / Kevin Wirya Valerian
2. 13524020 / Stevanus Agustaf Wongso
3. 13524023 / Jonathan Kris Wicaksono
4. 13524043 / Emilio Justin
5. 13524101 / Philipp Hamara

ORI - origami

1. Diagram Kelas

Di halaman setelah ini terdapat desain diagram kelas yang lengkap dengan atribut dan method penting yang sudah kami rancang berdasarkan spesifikasi tugas besar OOP. Untuk pengalaman membaca diagram yang lebih baik, dapat mengakses tautan berikut Class Diagram - Tugas Besar 1 OOP Origami


Origami
- board: Board&
- active_player_id: int
- players: vector<Player>&
- current_turn: int
- turn_order: vector<Player*>
- transaction_log : vector<LogEntry>
nextTurn()
checkWinCondition()
distributeSkillCards()
OutputFormatter
+ printBoard(in &b: Board)
+ printAksa()
+ printProperty(in p: Player&)
+ printLog(in log: vector<LogEntry>&)
+ printPlayerStatus(in p: Player&)
+ printAuction()
+ printWin(in pw: vector<Player>&)
Board
+ vector<Tile> Tiles
+ getTileByCode(in code: string) : Tile*
+ getTileByIndex(in index: int) : Tile*
+ getPropertiesByColorGroup(in color: string) : vector<PropertyTile*>
+ getColorGroupSize(color: string) : int
+ getTileIndex(code: string) : int
Bot
+ move() override
Player
curr_tile: int
balance : int
jail_counter : int
owned_properties : vector<PropertyTile*>
status: string
skill_used: boolean
discount_active: float
shield_active: boolean
jail_turns: int
virtual + move()
+ calculateNetWorth() : int
Dice
- die1: int
- die2: int
- double_count: int
throwDice(in count: int)
setDice(in x, y: int)
isDouble(): bool
getTotal(): int
FileManager
+ saveConfig()
+ loadConfig()
GameConfig
- go_salary : int
- jail_fine : int
- pph_flat : int
- pph_percentage : int
- phm_flat : int
- max_turn : int
- initial_balance : int
- railroad_rent : map<int, int>
- utility_multiplier : map<int, int>
LogEntry
- turn : int
- username : string
- action_type: string
- description: string
EventManager
win(in p: Player&)
bankrupt(in p: Player&)
auctionAll(in p: player&)
transferAssets(in p1: player&, p2: Player&)
ActionManager
+ buy(in &buyer: Player, in &tile: Tile)
+ payRent(in &buyer, owner: Player, tile: Tile)
+ payTax(in &t : Tile)
+ mortgage()
+ build()
+ auction()
+ festival()
+ getCard()
+ useSkillCard()
+ dropSkillCard()
+ goToJail()
+ compensate(in player: &Player, tile: &Tile)
+ liquidate(in player: &Player, tile: &Tile)
+ calculateMaxLiquidation(in player: &Player)
GeneralException()
InsufficientMoneyException()
UnablePayPropertyException()
UnablePayPPHTaxException()
UnablePayPBMTaxException()
UnablePayRentException()
UnablePayBuildException()
NoAktaException()
UnablePawnException()
UnablePawnExistBuildingException()
UnablePawnNoPropertyException()
UnableBuildException()
InsufficientMonopolyTermsException()
BuildingDifferenceException()
UndesirableException()
NoPropertyException()
UnableCompensateException()
FestivalException()
InvalidPropertyCodeException()
PropertyNotYoursException()
BankruptcyException()
BankruptCauseBankException()
BankruptCausePlayerException()
SkillCardException()
LimitOnlyOneException()
AfterDiceRollException()
Tile
- code: string
- id: string
- name: string
- type: string
PropertyTile
- rent_price : int []
- mortgage_value : int
- level : int
- owner : *Player
- mortgage_status : boolean
- color_group : string
- buy_price : int
- tile_cost : int
- upgrade_house_cost : int
- upgrade_hotel_cost : int
- festival_multiplier : int
- festival_duration : int
+ calculateRent()
+ isMortgaged()
+ isMonopolized()
ActionTile
virtual + onLanded(in p: Player&)
CardTile
+ onLanded(in p: Player&) override
ChanceTile
+ onLanded(in p: Player&) override
CommunityChestTile
+ onLanded(in p: Player&) override
FestivalTile
+ onLanded(in p: Player&) override
TaxTile
+ onLanded(in p: Player&) override
SpecialTile
virtual + onLanded(in p: Player&)
StartTile
+ onLanded(in p: Player&) override
JailTile
+ onLanded(in p: Player&) override
FreeParkingTile
+ onLanded(in p: Player&) override
GoJailTile
+ onLanded(in p: Player&) override
CardDeck<T>
- draw_pile: vector<T*>
- discard_pile: vector<T*>
+ draw(): T*
+ discard(): T*
+ shuffle()
+ refillFromDiscard()
Card
virtual + action(in p : &Player)
ChanceCard
StepbackCard
+ action(in p : &Player) override
NearestStreetCard
+ action(in p : &Player) override
JailCard
+ action(in p : &Player) override
CommunityChestCard
HappyBirthdayCard
+ action(in p: &Player) override
SickCard
+ action(in p : &Player) override
LegislativeCard
+ action(in p : &Player) override
SpecialPowerCard
MoveCard
value : int
+ action(in p: &Player) override
TeleportCard
+ action(in p : &Player) override
DiscountCard
value : int
+ action(in p: &Player) override
LassoCard
+ action(in p : &Player) override
ShieldCard
+ action(in p: &Player) override
DemolitionCard
+ action(in p : &Player) override

2. Deskripsi Kelas

2.1. Kelas Origami

Kelas pusat yang menjalankan keseluruhan alur permainan. Atributnya mencakup board, players, active_player_id, current_turn, turn_order, dan transaction_log. Method utamanya nextTurn(), checkWinCondition(), dan distributeSkillCards(). Origami memiliki hubungan komposisi dengan Board, Player melalui vector<Player>, LogEntry melalui transaction_log, serta hubungan use dengan Dice, FileManager, dan OutputFormatter.

2.2. Kelas Player

Mengatur pergerakan player dan menyimpan seluruh state-nya (uang, posisi, properti, kartu skill, status jail). Player memiliki hubungan use dengan Dice dan use dengan CardDeck<T> (saat mengambil/menyimpan kartu). Player menjadi parent dari Bot melalui hubungan is-a.

2.2.1. Kelas Bot

Mengatur otomatisasi pengambilan keputusan bot. Memiliki hubungan is-a (inheritance) dengan Player, sehingga seluruh atribut dan method Player dapat dipakai ulang sambil meng-override perilaku decision making agar berjalan otomatis tanpa input pengguna.

2.3. Kelas Board

Menyimpan kumpulan petak game melalui atribut vector<Tile> dan menyediakan utilitas pencarian seperti getTileByCode(), getTileByIndex(), getPropertiesByColorGroup(), getColorGroupSize(), dan getTileIndex(). Board memiliki hubungan komposisi dengan Tile karena Tile tidak dapat hidup tanpa Board. Board juga digunakan (use) oleh ActionManager.

2.4. Kelas Dice

Menyimpan state dua dadu dan menghasilkan nilai lemparan. Dice memiliki hubungan use dua arah: digunakan oleh Player saat melempar, dan menggunakan ActionManager untuk memicu aksi lanjutan setelah pergerakan.

2.5. Kelas FileManager

Menangani seluruh I/O file (konfigurasi, save, load). Digunakan (use) oleh Origami untuk memuat state awal, dan oleh GameConfig untuk membaca parameter konfigurasi.

2.6. Kelas GameConfig

Menyimpan parameter konfigurasi global (jumlah pemain, modal awal, aturan rumah). Memiliki hubungan use dengan FileManager untuk memuat data, dan digunakan oleh ActionManager.

2.7. Kelas LogEntry

Merepresentasikan satu baris transaksi/aksi. LogEntry diagregasi (has-a) oleh Origami melalui atribut transaction_log: vector<LogEntry> untuk mendukung fitur history.

2.8. Kelas Tile

Kelas abstrak yang menjadi base class seluruh jenis petak. Tile memiliki dua subclass langsung melalui relasi is-a: PropertyTile dan ActionTile. Polymorphism pada Tile memungkinkan Board menyimpan koleksi Tile* tunggal namun tetap dapat memanggil perilaku berbeda saat player mendarat.

2.8.1. Kelas PropertyTile

Merepresentasikan petak properti yang dapat dibeli, dibangun, dan disewakan. Memiliki hubungan is-a dengan Tile.

2.8.2. Kelas ActionTile

Merepresentasikan petak yang memicu aksi khusus saat didarati. Memiliki hubungan is-a dengan Tile, dan menjadi parent (is-a) dari empat subclass: CardTile, FestivalTile, TaxTile, dan SpecialTile.

2.8.2.1. Kelas CardTile

Petak yang memicu pengambilan kartu. Memiliki hubungan is-a dengan ActionTile, dan menjadi parent (is-a) dari ChanceTile dan CommunityChestTile.

2.8.2.1.1. Kelas ChanceTile

Petak yang memicu pengambilan ChanceCard. Memiliki hubungan is-a dengan CardTile.

2.8.2.1.2. Kelas CommunityChestTile

Petak yang memicu pengambilan CommunityChestCard. Memiliki hubungan is-a dengan CardTile.

2.8.2.2. Kelas FestivalTile

Petak festival yang menggandakan sewa pada satu warna properti. Memiliki hubungan is-a dengan ActionTile.

2.8.2.3. Kelas TaxTile

Petak pajak (PPh / PBM). Memiliki hubungan is-a dengan ActionTile.

2.8.2.4. Kelas SpecialTile

Parent untuk seluruh petak spesial. Memiliki hubungan is-a dengan ActionTile, dan menjadi parent (is-a) bagi empat subclass: StartTile, JailTile, FreeParkingTile, dan GoJailTile.

2.8.2.4.1. Kelas StartTile

Petak Start tempat player mendapat gaji setiap melewatinya. Hubungan is-a dengan SpecialTile.

2.8.2.4.2. Kelas JailTile

Petak penjara tempat player ditahan. Hubungan is-a dengan SpecialTile.

2.8.2.4.3. Kelas FreeParkingTile

Petak parkir gratis tanpa efek aksi. Hubungan is-a dengan SpecialTile.

2.8.2.4.4. Kelas GoJailTile

Petak yang langsung memindahkan player ke JailTile. Hubungan is-a dengan SpecialTile.

2.9. Kelas CardDeck

Kelas template yang mengelola tumpukan kartu generik (draw, shuffle, return). Memiliki hubungan use dengan Card (sebagai parameter generic T), digunakan oleh ActionManager dan Player. Penggunaan template memungkinkan satu kelas dipakai untuk seluruh jenis kartu tanpa duplikasi.

2.10. Kelas Card

Base class abstrak seluruh kartu dalam game. Card memiliki tiga subclass langsung melalui hubungan is-a: ChanceCard, CommunityChestCard, dan SpecialPowerCard. Polymorphism memungkinkan setiap kartu mengimplementasikan method use()-nya sendiri.

2.10.1. Kelas ChanceCard

Subclass dari Card. Memiliki hubungan is-a dengan Card, dan menjadi parent (is-a) dari NearestStreetCard, StepbackCard, dan JailCard.

2.10.1.1. Kelas StepbackCard

Kartu yang memundurkan posisi player. Hubungan is-a dengan ChanceCard.

2.10.1.2. Kelas NearestStreetCard

Kartu yang memindahkan player ke properti terdekat. Hubungan is-a dengan ChanceCard.

2.10.1.3. Kelas JailCard

Kartu Get-Out-of-Jail. Hubungan is-a dengan ChanceCard.

2.10.2. Kelas CommunityChestCard

Subclass dari Card. Memiliki hubungan is-a dengan Card, dan menjadi parent (is-a) dari HappyBirthdayCard, SickCard, dan LegislativeCard.

2.10.2.1. Kelas HappyBirthdayCard

Kartu yang membuat semua player lain memberi uang ke pemegang kartu. Hubungan is-a dengan CommunityChestCard.

2.10.2.2. Kelas SickCard

Kartu yang mengharuskan player membayar biaya kesehatan. Hubungan is-a dengan CommunityChestCard.

2.10.2.3. Kelas LegislativeCard

Kartu yang memberi efek legislatif (perubahan aturan sementara). Hubungan is-a dengan CommunityChestCard.

2.10.3. Kelas SpecialPowerCard

Subclass dari Card. Memiliki hubungan is-a dengan Card, dan menjadi parent (is-a) bagi seluruh kartu skill: MoveCard, TeleportCard, ShieldCard, LassoCard, DiscountCard, dan DemolitionCard.

2.10.3.1. Kelas MoveCard

Kartu skill yang memindahkan player sejumlah langkah tertentu. Hubungan is-a dengan SpecialPowerCard.

2.10.3.2. Kelas DiscountCard

Kartu skill yang memberi diskon pada transaksi properti. Hubungan is-a dengan SpecialPowerCard.

2.10.3.3. Kelas ShieldCard

Kartu skill yang melindungi player dari satu efek negatif. Hubungan is-a dengan SpecialPowerCard.

2.10.3.4. Kelas TeleportCard

Kartu skill yang memindahkan player ke petak tujuan tertentu. Hubungan is-a dengan SpecialPowerCard.

2.10.3.5. Kelas LassoCard

Kartu skill yang menarik player lain ke petak tertentu. Hubungan is-a dengan SpecialPowerCard.

2.10.3.6. Kelas DemolitionCard

Kartu skill yang menghancurkan bangunan player lain. Hubungan is-a dengan SpecialPowerCard.

2.11. Kelas ActionManager

Memusatkan seluruh aksi turn-level: buy(), payRent(), payTax(), mortgage(), build(), auction(), festival(), getCard(), useSkillCard(), dropSkillCard(), goToJail(), compensate(), liquidate(), dan calculateMaxLiquidation(). Memiliki hubungan use dengan Board (untuk query petak), CardDeck<T> (untuk menarik kartu), dan GameConfig (untuk parameter aturan). ActionManager melempar exception turunan dari GeneralException ketika aksi tidak valid.

2.12. Kelas EventManager

Menangani event game-level: win(), bankrupt(), auctionAll(), dan transferAssets(). Memiliki hubungan use dengan GeneralException untuk melempar exception saat event tertentu terpicu. Pemisahan dengan ActionManager dilakukan agar event game-level tidak bercampur dengan aksi turn-level.

2.13. Kelas OutputFormatter

Menyimpan template output yang berkaitan dengan game. Bersifat utility dan dapat dipanggil dari Origami maupun manager-manager lain agar tampilan ke layar tetap konsisten dan terpusat.

2.14. Kelas GeneralException

Base class abstrak untuk seluruh exception khusus game. GeneralException memiliki 14 subclass langsung melalui hubungan is-a, dan sebagian di antaranya (FestivalException, BankruptException, SkillCardException, UnablePawnException, dan UnableBuildException) juga menjadi parent dari subclass yang lebih spesifik sehingga membentuk hierarki dua tingkat. ActionManager dan EventManager

menggunakan (use) hierarki ini untuk melempar error spesifik. Hierarki dua tingkat memungkinkan kode pemanggil melakukan catch granular untuk kasus spesifik maupun catch generik di level kategori atau di level GeneralException.

2.14.1. Kelas InsufficientMoneyException

Dilempar saat player tidak memiliki cukup uang untuk transaksi apapun. Hubungan is-a dengan GeneralException.

2.14.2. Kelas UnablePayPropertyException

Dilempar saat player tidak mampu membayar harga properti. Hubungan is-a dengan GeneralException.

2.14.3. Kelas UnablePayPPHTaxException

Dilempar saat player tidak mampu membayar pajak PPh. Hubungan is-a dengan GeneralException.

2.14.4. Kelas UnablePayPBMTaxException

Dilempar saat player tidak mampu membayar pajak PBM. Hubungan is-a dengan GeneralException.

2.14.5. Kelas UnablePayRentException

Dilempar saat player tidak mampu membayar sewa. Hubungan is-a dengan GeneralException.

2.14.6. Kelas UnablePayBuildException

Dilempar saat player tidak mampu membayar harga pembangunan. Hubungan is-a dengan GeneralException.

2.14.7. Kelas NoAktaException

Dilempar saat player tidak memiliki akta properti yang diperlukan. Hubungan is-a dengan GeneralException.

2.14.8. Kelas UndesirableException

Dilempar untuk kondisi error tidak diinginkan yang spesifik. Hubungan is-a dengan GeneralException.

2.14.9. Kelas NoPropertyException

Dilempar saat player tidak memiliki properti sama sekali untuk aksi tertentu. Hubungan is-a dengan GeneralException.

2.14.10. Kelas UnableCompensateException

Dilempar saat kompensasi pembayaran tidak dapat dilakukan. Hubungan is-a dengan GeneralException.

2.14.11. Kelas FestivalException

Dilempar terkait pelanggaran aturan festival. Memiliki hubungan is-a dengan GeneralException, dan menjadi parent (is-a) dari InvalidPropertyCodeException dan PropertyNotYoursException.

2.14.11.1. Kelas InvalidPropertyCodeException

Dilempar saat kode properti yang dimasukkan saat festival tidak valid. Memiliki hubungan is-a dengan FestivalException.

2.14.11.2. Kelas PropertyNotYoursException

Dilempar saat player mencoba menggunakan festival pada properti milik orang lain. Memiliki hubungan is-a dengan FestivalException.

2.14.12. Kelas BankruptException

Menandai kondisi kebangkrutan player. Memiliki hubungan is-a dengan GeneralException, dan menjadi parent (is-a) dari BankruptCauseBankException dan BankruptCausePlayerException.

2.14.12.1. Kelas BankruptCauseBankException

Dilempar saat player bangkrut karena hutang ke bank. Memiliki hubungan is-a dengan BankruptException.

2.14.12.2. Kelas BankruptCausePlayerException

Dilempar saat player bangkrut karena hutang ke player lain. Memiliki hubungan is-a dengan BankruptException.

2.14.13. Kelas SkillCardException

Dilempar saat penggunaan skill card melanggar aturan umum. Memiliki hubungan is-a dengan GeneralException, dan menjadi parent (is-a) dari LimitOnlyOneException dan AfterDiceRollException.

2.14.13.1. Kelas LimitOnlyOneException

Dilempar saat player melebihi batas satu skill card per giliran. Memiliki hubungan is-a dengan SkillCardException.

2.14.13.2. Kelas AfterDiceRollException

Dilempar saat skill card digunakan pada timing setelah lemparan dadu yang tidak diizinkan. Memiliki hubungan is-a dengan SkillCardException.

2.14.14. Kelas UnablePawnException

Dilempar saat properti tidak dapat digadaikan. Memiliki hubungan is-a dengan GeneralException, dan menjadi parent (is-a) dari UnablePawnExistBuildingException dan UnablePawnNoPropertyException.

2.14.14.1. Kelas UnablePawnExistBuildingException

Dilempar saat gadai gagal karena masih ada bangunan di properti tersebut. Memiliki hubungan is-a dengan UnablePawnException.

2.14.14.2. Kelas UnablePawnNoPropertyException

Dilempar saat gadai gagal karena player tidak memiliki properti. Memiliki hubungan is-a dengan UnablePawnException.

2.14.15. Kelas UnableBuildException

Dilempar saat pembangunan tidak dapat dilakukan. Memiliki hubungan is-a dengan GeneralException, dan menjadi parent (is-a) dari InsufficientMonopolyTermsException dan BuildingDifferenceException.

2.14.15.1. Kelas InsufficientMonopolyTermsException

Dilempar saat syarat monopoli warna belum terpenuhi untuk membangun. Memiliki hubungan is-a dengan UnableBuildException.

2.14.15.2. Kelas BuildingDifferenceException

Dilempar saat selisih jumlah bangunan antar properti satu warna melebihi batas yang diizinkan. Memiliki hubungan is-a dengan UnableBuildException.

3. Justifikasi

Desain class diagram kami mengikuti prinsip separation of concerns dengan memisahkan tiga lapisan utama: (1) lapisan data/state yang diwakili oleh Player, Board, Tile dan turunannya, Card dan turunannya, Dice, LogEntry, dan GameConfig; (2) lapisan service/logic yang diwakili oleh ActionManager, EventManager, CardDeck, FileManager, dan OutputFormatter; serta (3) lapisan orchestration yang dipegang oleh Origami. Pemisahan ini sengaja dipilih agar kelas entitas tetap ramping dan tidak menumpuk logika transaksi, sehingga lebih mudah di-test dan di-maintain.

Kelebihan desain ini:

- Penggunaan inheritance bertingkat pada Tile (Tile → ActionTile → CardTile → ChanceTile/CommunityChestTile, atau Tile → ActionTile → SpecialTile → StartTile/JailTile/dst.) serta pada Card (Card → ChanceCard/CommunityChestCard/SpecialPowerCard → kartu spesifik) memungkinkan polymorphism, sehingga penambahan jenis petak atau kartu baru tidak memerlukan perubahan besar pada Board atau ActionManager, cukup membuat subclass baru sesuai Open/Closed Principle
- Bot yang mewarisi Player memungkinkan reuse kode pergerakan tanpa duplikasi
- Penggunaan kelas template CardDeck<T> menghindari duplikasi logika tumpukan kartu untuk setiap jenis kartu
- Hierarki GeneralException dengan dua tingkat (general → kategori seperti FestivalException, BankruptException, SkillCardException, UnablePawnException, UnableBuildException → kasus spesifik) memungkinkan error handling yang granular sekaligus tetap dapat di-catch secara generik di level kategori
- Sentralisasi I/O di FileManager dan output di OutputFormatter membuat presentation logic terpisah dari business logic.

Kekurangan yang kami sadari:

- Hubungan dua arah antara Dice dan ActionManager berpotensi menimbulkan circular dependency yang perlu ditangani dengan forward declaration di C++
- Hierarki Tile memiliki empat level kedalaman (Tile → ActionTile → CardTile/SpecialTile → ChanceTile/StartTile/dll.) sehingga butuh disiplin dokumentasi
- Jumlah subclass exception yang banyak berpotensi membuat maintenance terasa verbose meskipun pengelompokan dua tingkat sudah membantu kerapian.

Kendala dalam pemilihan desain: tantangan terbesar adalah menentukan batas tanggung jawab antara ActionManager dan EventManager, kami akhirnya memutuskan ActionManager menangani aksi turn-level (dipicu keputusan player), sedangkan EventManager menangani peristiwa game-level (dipicu kondisi game seperti bangkrut atau menang). Kendala lain adalah memutuskan apakah Card perlu hierarki dalam atau cukup satu kelas dengan field type; kami memilih hierarki agar setiap kartu dapat memiliki implementasi action() berbeda secara polymorphic. Pemilihan template CardDeck<T> juga sempat didiskusikan dibandingkan membuat tiga deck terpisah, dan kami memilih template demi reuse kode.